MAHARISHI VIDYA MANDIR SR.SEC.SCHOOL POLACHERY

ACADEMIC YEAR 2026-2027

COMPUTER SCIENCE PROJECT

TOPIC : BANK MANAGEMENT USING MYSQL CONNECTIVITY

NAME: G.M Dharani Pethiah
CLASS: XII A1

INDEX

S.NO	TOPIC	PG.NO
1.	BONAFIDE CERTIFICATE	
2.	ACKNOWLEDGEMENT	
3.	INTRODUCTION	
4.	PROBLEM DEFINITION	
5.	PROBLEM ANALYSIS	
6.	SYSTEM CONFIGURATION	
7.	FUTURE ENHANCEMENT	
8.	PYTHON SOURCE CODE	
9.	OUTPUT CODE	
10.	CONCLUSION	
11.	BIBLIOGRAPHY	

INTRODUCTION

In today's digital world, banks and financial institutions rely heavily on computer-based systems to manage customer accounts and transactions efficiently. Manual maintenance of banking records is time-consuming, prone to errors, and difficult to manage when dealing with a large number of customers. Therefore, computerized banking systems have become essential for ensuring accuracy, speed, and security in banking operations.

The "Bank Management System using Python and MySQL Connectivity" is a software application developed to perform basic banking activities in an organized and user-friendly manner. This project is designed using Python programming language, MySQL database, and Tkinter graphical user interface (GUI). The system allows users to create new accounts, deposit money, withdraw money, check account balances, view all customer records, and delete accounts when required.

The project demonstrates the practical implementation of database management concepts and Python programming techniques. MySQL is used to store and manage customer information permanently, while Python handles the business logic and communication with the database. Tkinter provides an interactive graphical interface that makes the application easy to use.

This project helps in understanding the concepts of database connectivity, CRUD operations (Create, Read, Update, Delete), data validation, exception handling, and GUI development. It also highlights the importance of information technology in modern banking systems. The Bank Management System serves as a simple yet effective example of how software applications can automate routine tasks, improve efficiency, and reduce human errors in real-world organizations.

PROBLEM DEFINITION

Managing bank account records manually is a difficult and time-consuming task. Traditional record-keeping methods may lead to errors in calculations, loss of data, duplication of records, and delays in processing customer transactions. As the number of customers increases, maintaining accurate information becomes even more challenging.

The main problem addressed by this project is the need for an efficient and reliable system to manage banking operations digitally. The system should be able to store customer details securely, perform transactions such as deposits and withdrawals, display account balances, and update records automatically without human errors.

To solve this problem, a Bank Management System has been developed using Python, MySQL, and Tkinter. The application provides a user-friendly interface and allows easy management of customer accounts. It ensures accurate record maintenance, quick transaction processing, and secure storage of data. The project demonstrates how database connectivity and programming can be used to automate banking activities and improve overall efficiency.

PROBLEM ANALYSIS

Banking institutions handle a large volume of customer data and financial transactions every day. Maintaining these records manually can be inefficient, time-consuming, and prone to errors. Tasks such as creating new accounts, updating balances after deposits or withdrawals, checking account details, and maintaining customer records require accuracy and proper organization. Manual systems often face problems such as data redundancy, calculation mistakes, misplaced records, and difficulty in retrieving information quickly.

A computerized banking system can overcome these limitations by automating routine banking operations. Before developing the project, the requirements of a basic banking system were analyzed. The system needed to provide facilities for account creation, money deposit, money withdrawal, balance enquiry, account deletion, and viewing all account records. It was also necessary to ensure that invalid transactions, such as withdrawing more money than the available balance, were prevented.

The project uses Python as the programming language due to its simplicity and powerful database connectivity features. MySQL is used as the backend database to store customer information securely and permanently. Tkinter is used to create a graphical user interface, making the system easy to operate for users without technical knowledge.

The analysis showed that integrating Python, MySQL, and Tkinter would provide an effective solution for managing banking records. The developed system reduces manual effort, improves accuracy, ensures faster data retrieval, and demonstrates the practical application of database management and programming concepts in a real-world scenario.

SYSTEM CONFIGURATION

HARDWARE CONFIGURATION :

- Processor : Intel Core i5 or equivalent
- Memory : 2 GB (32-bit), 4 GB (64-bit)
- Disk space : 1.5 GB of free disk space

SOFTWARE REQUIREMENTS :

- 1 GB RAM (2 GB+ recommended)
- 9-58 GB free hard disk space depending on edition and configuration, including space required for temporary files
- DVD-ROM drive (if installing from a Media Kit DVD)
- Basic GPU – Any vendor DirectX 9.0 class or better (Pixel Shader Level 2)
- Intel® Pentium® or compatible, 1.6 GHz minimum (2GHz+ recommended)
- 1024x768 or higher-resolution monitor
- MOUSE OR OTHER POINTING DEVICE

FUTURE ENHANCEMENT

1) **User Login Authentication** – Add a secure username and password system to prevent unauthorized access to banking records.

2) **Fund Transfer Facility** – Enable customers to transfer money directly from one account to another within the system.

3) **Transaction History** – Maintain a complete record of all deposits, withdrawals, and transfers for future reference.

4) **Interest Calculation** – Include automatic calculation of interest for savings accounts based on predefined rates.

5) **Account Update Feature** – Allow users to modify customer details such as name, phone number, and address when required.

6) **Report Generation** – Generate and print account statements, transaction reports, and customer summaries in PDF or other formats.

7) **Enhanced Security Measures** – Implement data encryption, OTP verification, and backup mechanisms to improve the security and reliability of the system.

PYTHON SOURCE CODE

```
# Simple Bank Management System
# Class 12 NCERT - Python + MySQL + Tkinter

import tkinter as tk
from tkinter import messagebox
import mysql.connector

# ── Database Connection ──────────────────────────
def connect():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="1234",
        database="bank"
    )

def setup():
    con = mysql.connector.connect(
        host="localhost",
        user="root",
        password="1234"
    )
    cur = con.cursor()
    cur.execute("CREATE DATABASE IF NOT EXISTS bank")
    con.database = "bank"
    cur.execute("""
        CREATE TABLE IF NOT EXISTS accounts (
            accno    INT PRIMARY KEY,
            name     VARCHAR(50),
            balance  FLOAT
```

```
        )
    """)
    con.commit()
    cur.close()
    con.close()


# ── Functions ──────────────────────────────────────────

def create_account():
    accno  = e_accno.get()
    name    = e_name.get()
    balance = e_balance.get()

    if accno == "" or name == "" or balance == "":
        messagebox.showerror("Error", "All fields are required!")
        return

    try:
        con = connect()
        cur = con.cursor()
        cur.execute("INSERT INTO accounts VALUES (%s, %s, %s)",
                    (int(accno), name, float(balance)))
        con.commit()
        cur.close(); con.close()
        messagebox.showinfo("Success", "Account Created Successfully!")
        clear()
        view_all()
    except Exception as e:
        messagebox.showerror("Error",  str(e))

def deposit():
    accno  = e_accno.get()
    amount = e_balance.get()

    if accno == "" or amount == "":
        messagebox.showerror("Error", "Enter Account No and Amount!")
        return
```

```
    try:
        con = connect()
        cur = con.cursor()
        cur.execute("SELECT * FROM accounts WHERE accno=%s",
(int(accno),))
        row = cur.fetchone()
        if row is None:
            messagebox.showerror("Error", "Account not found!")




return
        new_bal = row[2] + float(amount)
        cur.execute("UPDATE accounts SET balance=%s WHERE
accno=%s",
                    (new_bal, int(accno)))
        con.commit()
        cur.close(); con.close()
        messagebox.showinfo("Success", f"Deposited! New Balance:
{new_bal}")
        clear()
        view_all()
    except Exception as e:
        messagebox.showerror("Error", str(e))

def withdraw():
    accno  = e_accno.get()
    amount = e_balance.get()

    if accno == "" or amount == "":
        messagebox.showerror("Error", "Enter Account No and Amount!")
        return


    try:
        con = connect()
        cur = con.cursor()
        cur.execute("SELECT * FROM accounts WHERE accno=%s",
(int(accno),))
        row = cur.fetchone()
        if row is None:
            messagebox.showerror("Error", "Account not found!")
```

```
            return
        if row[2] < float(amount):
            messagebox.showerror("Error", "Insufficient Balance!")
            return
        new_bal = row[2] - float(amount)
        cur.execute("UPDATE accounts SET balance=%s WHERE
accno=%s",
                    (new_bal, int(accno)))
        con.commit()
        cur.close(); con.close()
        messagebox.showinfo("Success", f"Withdrawn! New Balance:



{new_bal}")
        clear()
        view_all()
    except Exception as e:
        messagebox.showerror("Error", str(e))

def check_balance():

accno = e_accno.get()
    if accno == "":
        messagebox.showerror("Error", "Enter Account Number!")
        return
    try:
        con = connect()
        cur = con.cursor()
        cur.execute("SELECT * FROM accounts WHERE accno=%s",
(int(accno),))
        row = cur.fetchone()
        cur.close(); con.close()
        if row:
            messagebox.showinfo("Balance", f"Name : {row[1]}\nBalance:
Rs. {row[2]}")
        else:
            messagebox.showerror("Error", "Account not found!")
    except Exception as e:
        messagebox.showerror("Error", str(e))
```

```
def delete_account():
    accno = e_accno.get()
    if accno == "":
        messagebox.showerror("Error", "Enter Account Number!")
        return
    confirm = messagebox.askyesno("Confirm", "Delete this account?")
    if not confirm:
        return
    try:
        con = connect()
        cur = con.cursor()
        cur.execute("DELETE FROM accounts WHERE accno=%s",
(int(accno),))




con.commit()
        cur.close(); con.close()
        messagebox.showinfo("Deleted", "Account deleted.")
        clear()
        view_all()
    except Exception as e:
        messagebox.showerror("Error", str(e))

def view_all():
    listbox.delete(0, tk.END)
    try:
        con = connect()
        cur = con.cursor()
        cur.execute("SELECT * FROM accounts")
        rows = cur.fetchall()
        cur.close(); con.close()
        listbox.insert(tk.END, f"{'AccNo':<10} {'Name':<20}
{'Balance':>10}")
        listbox.insert(tk.END, "-" * 42)
        for row in rows:
            listbox.insert(tk.END, f"{row[0]:<10} {row[1]:<20}
{row[2]:>10.2f}")
    except Exception as e:
        messagebox.showerror("Error", str(e))
```

```
def clear():
    e_accno.delete(0, tk.END)
    e_name.delete(0, tk.END)
    e_balance.delete(0, tk.END)

#
————————————————————————————————————————————————
———————————————— TKINTER DESIGN ————————————————
————————————————————————————————————————————————
setup()

win = tk.Tk()
win.title("Bank Management System")
win.geometry("600x520")
win.resizable(False, False)
win.config(bg="#f0f4f8")




# Title
tk.Label(win, text="Bank Management System",
         font=("Arial", 18, "bold"),
         bg="#2c3e50", fg="white",
         pady=10).pack(fill="x")

# Input Frame
frame = tk.Frame(win, bg="#f0f4f8", pady=10)
frame.pack()

tk.Label(frame, text="Account No :", bg="#f0f4f8",
         font=("Arial", 11)).grid(row=0, column=0, padx=10, pady=6,
sticky="e")

e_accno = tk.Entry(frame, font=("Arial", 11), width=22, relief="solid")
e_accno.grid(row=0, column=1, padx=10, pady=6)

tk.Label(frame, text="Name :", bg="#f0f4f8",
         font=("Arial", 11)).grid(row=1, column=0, padx=10, pady=6,
sticky="e")
```

```
e_name = tk.Entry(frame, font=("Arial", 11), width=22, relief="solid")
e_name.grid(row=1, column=1, padx=10, pady=6)

tk.Label(frame, text="Amount / Balance :", bg="#f0f4f8",
         font=("Arial", 11)).grid(row=2, column=0, padx=10, pady=6,
sticky="e")
e_balance = tk.Entry(frame, font=("Arial", 11), width=22, relief="solid")
e_balance.grid(row=2, column=1, padx=10, pady=6)

# Buttons Frame
bf = tk.Frame(win, bg="#f0f4f8")
bf.pack(pady=8)

btn_style = {"font": ("Arial", 10, "bold"), "width": 16, "pady": 5, "relief":
"flat"}

tk.Button(bf, text="Create Account", bg="#27ae60", fg="white",
          command=create_account, **btn_style).grid(row=0, column=0,
padx=6, pady=4)




tk.Button(bf, text="Deposit",         bg="#2980b9", fg="white",
          command=deposit,          **btn_style).grid(row=0, column=1,
padx=6, pady=4)

tk.Button(bf, text="Withdraw",        bg="#e67e22", fg="white",
          command=withdraw,         **btn_style).grid(row=0, column=2,
padx=6, pady=4)

tk.Button(bf, text="Check Balance",  bg="#8e44ad", fg="white",
          command=check_balance,  **btn_style).grid(row=1, column=0,
padx=6, pady=4)

tk.Button(bf, text="Delete Account", bg="#c0392b", fg="white",
          command=delete_account,  **btn_style).grid(row=1, column=1,
padx=6, pady=4)

tk.Button(bf, text="View All",        bg="#2c3e50", fg="white",
```

```
command=view_all,          **btn_style).grid(row=1, column=2,
padx=6, pady=4)

tk.Button(bf, text="Clear",            bg="#7f8c8d", fg="white",
          command=clear,           **btn_style).grid(row=2, column=1,
padx=6, pady=4)

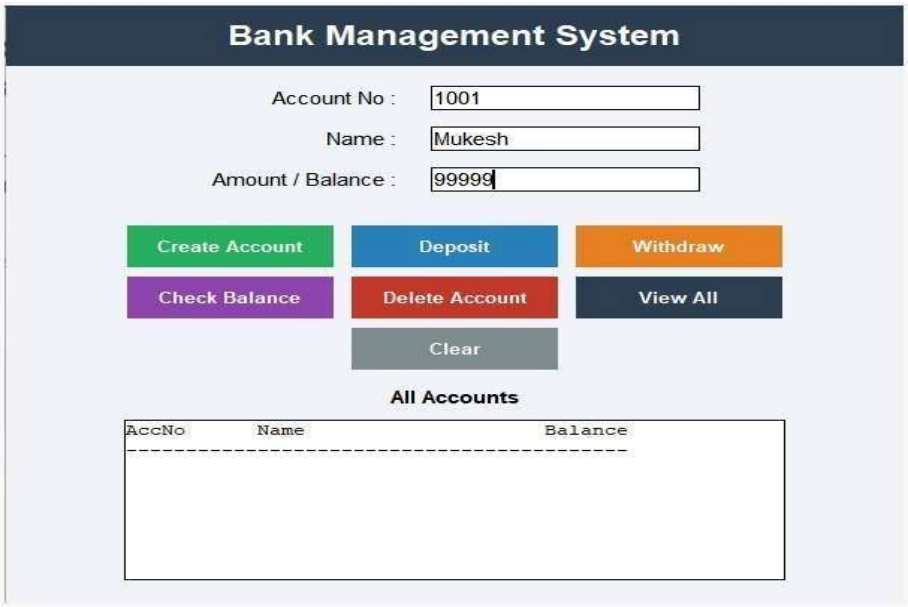
# Listbox
tk.Label(win, text="All Accounts", font=("Arial", 11, "bold"),
         bg="#f0f4f8").pack()

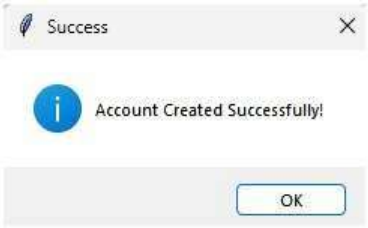
listbox = tk.Listbox(win, font=("Courier", 10), width=55, height=8,
                     relief="solid", bd=1)
listbox.pack(padx=20, pady=6)

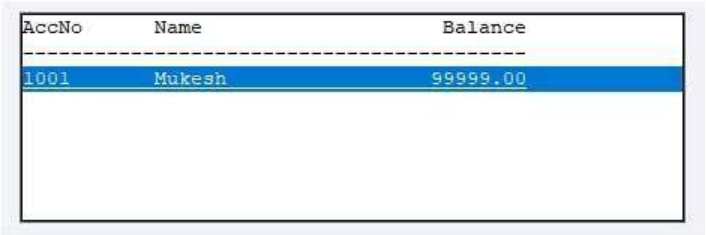
view_all()
win.mainloop()
```

OUTPUT

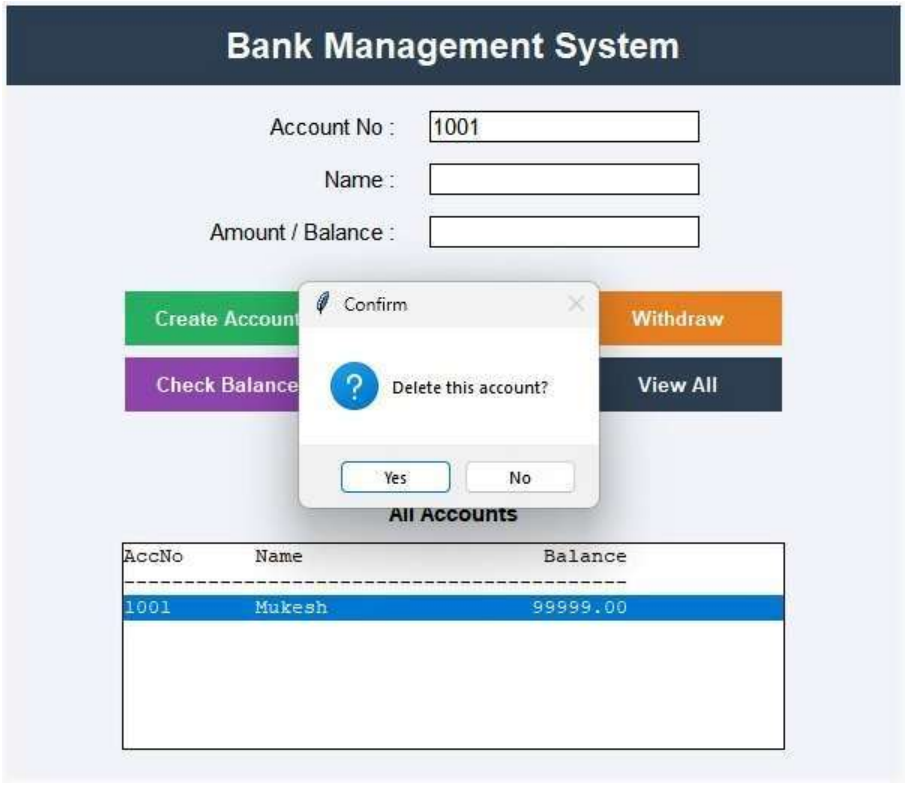
CREATE AN ACCOUNT -

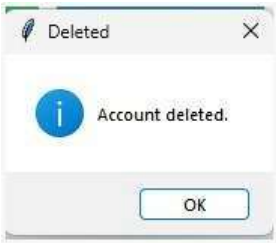


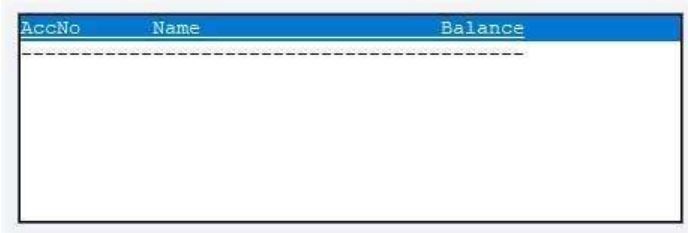




DELTE AN ACCOUNT -

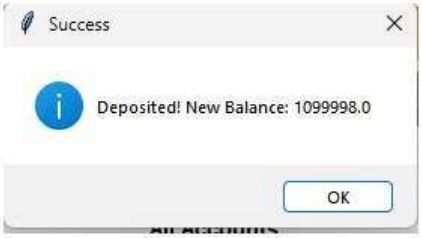






DEPOSIT AMOUNT -

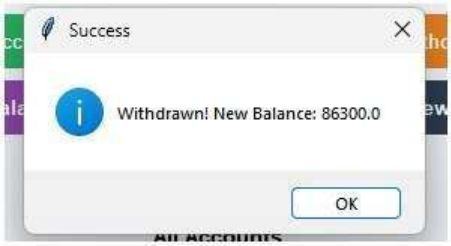




AccNo	Name	Balance
1001	Mukesh	1100000.00
1002	Dharani	94300.00
1003	Parthiv	47086.00

WITHDRAWING AMOUNT -

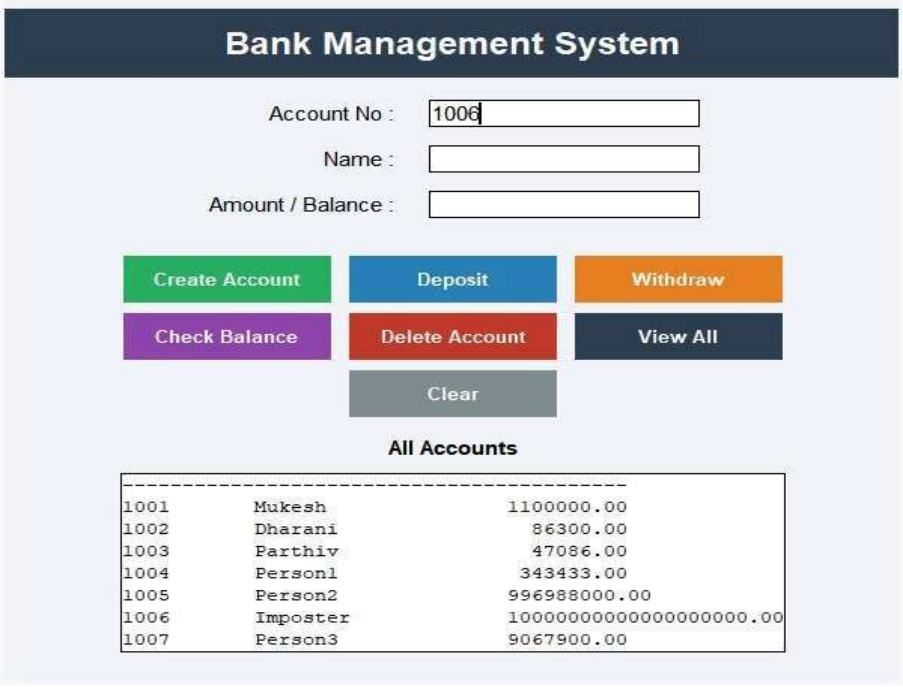




All Accounts

AccNo	Name	Balance
1001	Mukesh	1100000.00
1002	Dharani	86300.00
1003	Parthiv	47086.00

CHECK BALANCE -





CONCLUSION

The Bank Management System developed using Python, MySQL, and Tkinter successfully demonstrates the practical application of programming and database management concepts. The system provides an efficient way to manage banking operations such as account creation, deposits, withdrawals, balance enquiries, account deletion, and viewing customer records. By automating these tasks, the project reduces manual effort, improves accuracy, and ensures faster access to information.

The use of MySQL enables secure and organized storage of customer data, while Python handles the application logic effectively. The Tkinter graphical user interface makes the system user-friendly and easy to operate. Throughout the development of this project, valuable knowledge was gained in database connectivity, SQL queries, GUI design, error handling, and CRUD operations.

Overall, the project achieves its objectives and serves as a simple yet effective banking application. It highlights the importance of computerization in modern banking systems and provides a strong foundation for developing more advanced financial management applications in the future.

BIBLIOGRAPHY

1. **NCERT Computer Science (Class XII)** – For Python and database connectivity concepts.

2. **Python.org** – For Python syntax and programming references.

3. **MySQL Documentation (mysql.com)** – For SQL commands and database operations.

4. **W3Schools.com** – For learning SQL queries and Python-MySQL integration.

5. **Classroom Notes and Practical Files** – For additional help and examples related to database connectivity.

LINKS:

- HTTPS://www.Python.ORG/
- HTTPS://WWW.MYSQL.COM/
- HTTPS://KVCODERS.IN